

Avalon-MM SD Card Controller IP Core

User Guide

IP core	<code>altera_avalon_mm_sdcard_controller</code>
Core version	1.0
Document version	1.0
Date	September 2026
Target	Intel FPGA / Quartus Prime, Platform Designer

Contents

1. About this core	4
1.1 Features	4
1.2 What it is for	4
1.3 Status	4
2. Getting started	5
2.1 Adding the component	5
2.2 Connecting it	5
2.3 Pin assignment	5
2.4 A first transfer	6
3. Functional description	7
3.1 The shape of an operation	7
3.2 Multi-block transfers	9
3.3 Timeouts	9
3.4 Soft resets	9
3.5 CRC	9
4. Parameters	10
4.1 On MAX_BLOCK_BYTES	10
4.2 On USE_DMA	10
5. Registers	11
5.1 CTRL (0x00)	11
5.2 STATUS (0x04)	12
5.3 IRQ_STATUS (0x0C) and IRQ_ENABLE (0x08)	12
5.4 CMD (0x1C)	13
5.5 ERR_INFO (0x3C)	13
6. Signals	14
6.1 Clock and reset	14
6.2 csr — Avalon-MM slave	14
6.3 m0 — Avalon-MM master (USE_DMA = 1)	14
6.4 sd — conduit	15
7. Software	16
7.1 Capacity, and why the CSD parse is separately tested	16
7.2 PIO mode	16
8. Performance	17
8.1 Measured	17
8.2 Where it comes from	17
8.3 What the DMA is and is not for	17

9. Verification	18
9.1 The configuration sweep	18
9.2 Assertions, and proving they are alive	18
9.3 Questa	19
10. Limitations and known gaps	20
11. Revision history	21

1. About this core

1.1 Features

- SD, SDHC and SDXC cards in **SPI mode** (Physical Layer Simplified Specification v4.10, chapter 7)
- Hardware link layer: command framing, CRC7 generation, CRC16 generation and checking, token recognition, data-response decoding, busy detection
- **Multi-block streaming in hardware**, including the automatic CMD12 stop for multi-block reads
- **Optional bursting DMA** (`m0`) that moves block data to and from memory without CPU involvement — or PIO through a register window when you have no master to spare
- Run-time SPI clock divider covering the 400 kHz identification rate and full speed from the same build
- Bounded waits on every phase, reporting **which** phase failed
- Split soft resets: reset the data path without losing card identification
- Nios II HAL driver with `alt_sdcard_*` API

1.2 What it is for

Bulk block transfer to and from an SD card from an FPGA, with a CPU that has other things to do. The hardware handles everything between "here is a command" and "the block is in memory": framing, CRCs, the tokens, the waiting, and the several places where the card is allowed to take an unbounded amount of time.

Card *protocol* — the initialisation dance, capacity classes, the CSD parse — lives in the driver, not the RTL. That split is deliberate. Initialisation is a sequence of ordinary commands with software-visible decisions in it; putting it in hardware buys nothing and makes every future card quirk an RTL change.

1.3 Status

This core has never run on a board. It has not been tested against a physical SD card, and no timing closure has been demonstrated on real hardware. What it has is:

Testbenches	3 — shifter, FIFO, full core
Distinct checks	57
Configurations swept	5 (<code>dma</code> , <code>pio</code> , <code>sdsc</code> , <code>tight</code> , <code>noburst</code>)
Bound SVA assertions	24, plus 5 cover points
Assertion fault injections	3 — each required to be caught
Platform Designer component checks	22
HAL driver checks	3
Lint configurations	10
Documentation claims re-derived from source	see <code>doc/tools/check_facts.py</code>

The card model is derived from the specification, not from a real card. Real cards deviate from the specification in ways a model written from the same document cannot predict — that is precisely the risk that remains, and it is not a small one.

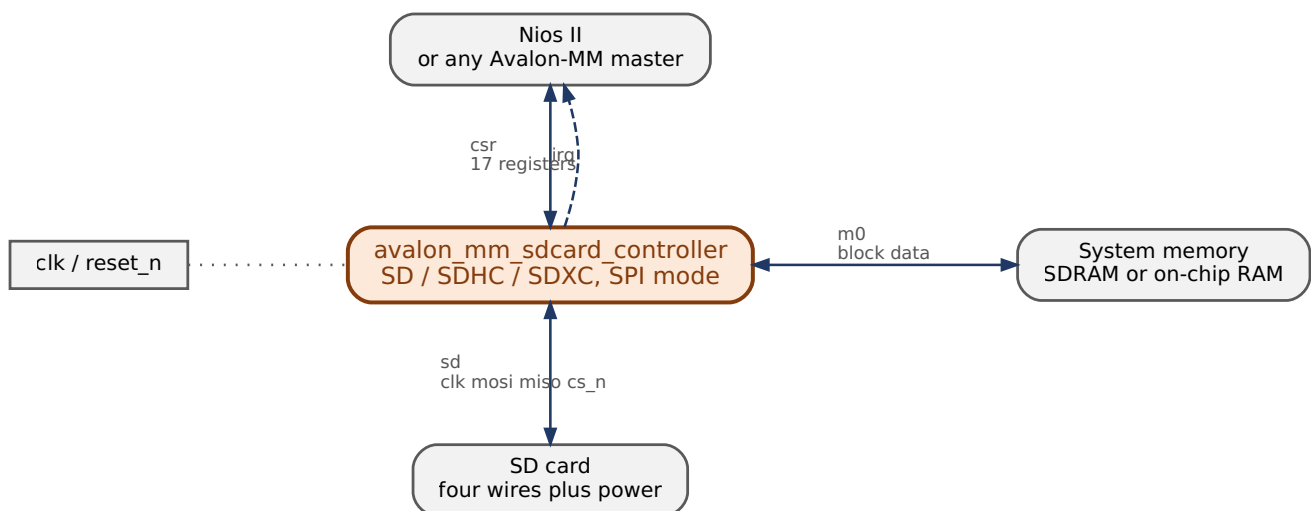
2. Getting started

2.1 Adding the component

Point Platform Designer at the directory containing `altera_avalon_mm_sdc_card_controller_hw.tcl`, then add **Avalon-MM SD Card Controller (SPI)** from *Memory Interfaces and Controllers / Custom*.

2.2 Connecting it

Interface	Connect to
<code>clock</code> , <code>reset</code>	Your system clock and reset
<code>csr</code>	The CPU's data master
<code>m0</code>	The memory the data lands in — SDRAM, on-chip RAM. Only present when <code>USE_DMA = 1</code>
<code>irq</code>	The CPU's interrupt receiver
<code>sd</code>	Export to the top level, then to pins



`m0` bursts. Platform Designer inserts a burst adapter wherever it meets a slave that bursts less, so `M0_BURST_WIDTH = 1` is a supported configuration rather than a degraded one — at SPI rates the memory side is never the bottleneck.

2.3 Pin assignment

Four signals plus two optional ones. In SPI mode the SD card's pins carry different meanings from their names in native mode:

Conduit	SD card pin	Notes
<code>sd_clk</code>	CLK (pin 5)	
<code>sd_mosi</code>	CMD (pin 2)	
<code>sd_miso</code>	DAT0 (pin 7)	Needs a pull-up. The card releases the line and it must idle high
<code>sd_cs_n</code>	DAT3/CS (pin 1)	Needs a pull-up during power-up, or the card may come up in native mode
<code>sd_cd_n</code>	socket switch	Optional
<code>sd_wp_n</code>	socket switch	Optional

DAT1 and DAT2 are unused in SPI mode but should be pulled up rather than left floating.

2.4 A first transfer

```
#include "altera_avalon_mm_sdcard_controller.h"

alt_sdcard_dev dev;
alt_u32          buf[128];          /* one 512-byte block */

if (alt_sdcard_init(&dev, SDCARD_0_BASE, ALT_CPU_FREQ) != 0)
    return -1;                      /* no card, or it refused */

if (alt_sdcard_read(&dev, 0, buf, 1) != 0) /* LBA 0, one block */
    return -1;
```

`alt_sdcard_init()` runs the identification sequence at 400 kHz, works out the card's capacity class, reads the CSD, and then raises the clock. Everything after it addresses the card in **512-byte blocks** regardless of class; the driver converts to a byte address for standard-capacity cards.

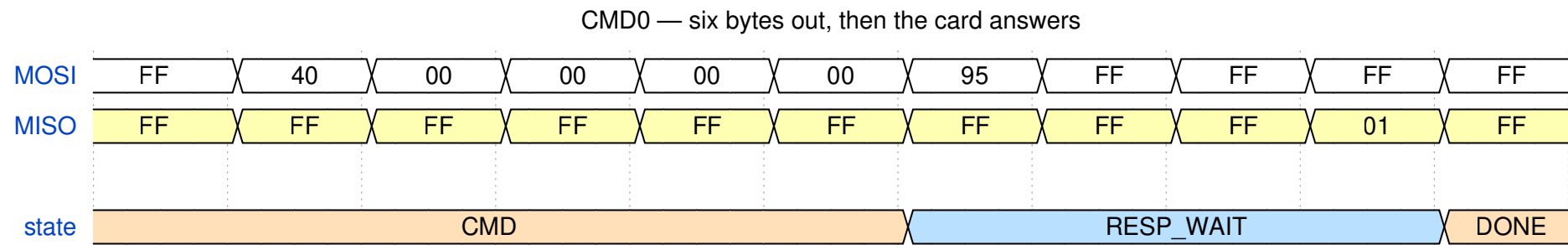
3. Functional description

3.1 The shape of an operation

The sequencer is a 20-state machine. Every operation is a command, optionally followed by a data phase:

1. **Pre-busy.** With `CS` high, the core free-runs `0xFF` until `MISO` is high. A card still programming a previous block holds the line low, and clocking it is how you find out.
2. **Command.** Six bytes: `0x40 | index`, four argument bytes, CRC7 shifted up one with the stop bit in bit 0.
3. **Response.** The core clocks `0xFF` until a byte arrives with bit 7 clear. `N_CR` is a **range** — 0 to 8 byte-times — not a fixed latency, so this is a poll and not a wait.
4. **Data phase**, if `CMD.DATA_EN` is set. Read: wait for the `0xFE` token, take `BLK_SIZE` bytes, then a two-byte CRC16. Write: send `0xFE`, the data, the CRC16, then read the data-response token and wait out the card's busy.
5. **Done.** `IRQ_STATUS` records what happened; `ERR_INFO` records the detail if it went wrong.

A recorded command and its response, one column per byte-time:



0x95 is the CRC7 of CMD0 with a zero argument — the constant that appears in every SD initialisation routine ever written. The card answered two byte-times after the frame; the specification allows anywhere from 0 to 8, which is why the core polls rather than waiting a fixed time.

3.2 Multi-block transfers

Set `BLK_COUNT` above 1 and `CMD.MULTI`, and the core streams blocks back-to-back without software in the loop. With `CMD.AUTO_STOP` set it also terminates the stream itself: **CMD12 for a multi-block read**, the 0xFD stop-token for a multi-block write.

Note the asymmetry, which is the specification's and not this core's: the 0xFE start token covers single-block read, single-block write *and* multiple-block read. Only multiple-block **write** uses its own 0xFC, and only that form has a stop token.

3.3 Timeouts

Every wait is bounded by `TIMEOUT`, and the bound is on **time without progress**, not total duration — so a long block never trips it and a stalled one does.

`ERR_INFO[27:24]` records which phase failed. This matters more than it looks: "the card never answered" and "the card answered and then never finished" need different recovery, and nothing else distinguishes them.

3.4 Soft resets

`CTRL` carries three reset bits, and they are separate for a reason:

Bit	Resets
<code>SRST_CMD</code>	The command path only
<code>SRST_DAT</code>	The data path and the FIFO only
<code>SRST_ALL</code>	Everything except configuration

Resetting the data path after a failed transfer leaves the card identified, so recovery does not mean redoing the whole 400 kHz initialisation. The regression checks this specifically, because a "reset" that quietly loses card state is a reset nobody can use.

3.5 CRC

	Polynomial	Initial value
CRC7, command frames	$x^7 + x^3 + 1$	0
CRC16, data blocks	$x^{16} + x^{12} + x^5 + 1$ (CCITT)	0x0000

That CRC16 initial value is the trap. "CCITT" in almost every other context means an initial value of 0xFFFF; SD specifies zero. A CRC16 seeded with 0xFFFF produces plausible-looking values that never match, and the failure looks exactly like a wiring fault.

Both are computed a byte at a time, because the shifter does not pause between bytes and a bit-serial CRC would have nowhere to do its work.

CRC7 on commands is unconditional even when `USE_CRC = 0`: CMD0 and CMD8 are validated by the card whatever CMD59 says.

4. Parameters

Parameter	Default	Range	Effect
FIFO_DEPTH_BYTES	1024	512, 1024, 2048, 4096, 8192	Decouples the shifter from memory. 1024 holds two blocks, so one can be on the wire while the other moves. 512 works but leaves no overlap
M0_BURST_WIDTH	8	1–9	Maximum burst is $2^{(N-1)}$ beats. 8 gives 128 beats, exactly one 512-byte block. 1 means no bursting — supported, not degraded
CLKDIV_WIDTH	8	4–16	SPI clock is $\text{clk}/(2 \cdot \text{CLKDIV})$, set at run time. 8 bits spans $\text{clk}/2$ to $\text{clk}/510$
TIMEOUT_WIDTH	26	16–32	26 bits at 100 MHz is 0.67 s, covering the specification's 250 ms write-busy and 100 ms read-access limits with margin
MAX_BLOCK_BYTES	512	16, 64, 128, 256, 512	512 is the specification maximum, not a convention
CSR_ADDR_WIDTH	5	5–8	In words . The map occupies 17, so 5 is the minimum
ADDR_WIDTH	32	16–32	Byte address width of <code>m0</code> . Ignored when the DMA is off
USE_DMA	1	0, 1	Off removes <code>m0</code> entirely. Costs roughly 10–20% of a 100 MHz Nios II/f during a transfer and nothing in throughput
USE_CARD_DETECT	1	0, 1	Adds <code>sd_cd_n</code> / <code>sd_wp_n</code> and the insert/remove interrupts. Off reports a card always present and never write-protected
USE_CRC	1	0, 1	CRC16 on data blocks. A debugging aid, not a performance option — it costs nothing

4.1 On MAX_BLOCK_BYTES

512 bytes is fixed by Physical Layer 7.2.3 regardless of `READ_BL_LEN`, and SDHC/SDXC cards accept nothing else. Smaller values only make sense for a design that will never touch a high-capacity card, or one that only ever reads the 16-byte CSD and CID.

This is worth being explicit about because it is easy to assume `BLK_SIZE` is freely programmable. It is — on the *controller*. The **card** keeps its own block length, 512 until CMD16 changes it, and CMD16 does not change it on a high capacity card. Setting the controller to a smaller block than the card is sending desynchronises the transfer: the read fails its CRC16, the write never gets a data-response token, and the next command is swallowed as data.

4.2 On USE_DMA

Turning the DMA off is a legitimate choice for a system with no suitable memory target, or none to spare. What changes is the *deadline*: with no master keeping the buffer moving, software has to service the `DATA` window fast enough that the shifter is never starved, and the data-phase stall timeout is what catches it when it is not.

That path is tested rather than assumed — `p10` is one of the five configurations the regression sweeps, and it is the only one in which those stall timeouts can be reached at all.

5. Registers

All registers are 32 bits, word-addressed on `csr`. Read latency is 1.

Offset	Word	Name	Access	Purpose
0x00	0	CTRL	RW	Enable, CS control, CRC, DMA, soft resets
0x04	1	STATUS	RO	Busy flags, FIFO level, card presence, error summary
0x08	2	IRQ_ENABLE	RW	Mask, one bit per source
0x0C	3	IRQ_STATUS	RW1C	Pending events and errors
0x10	4	CLKDIV	RW	SPI clock = $\text{clk} / (2 \cdot \text{CLKDIV})$
0x14	5	TIMEOUT	RW	Cycles without progress before giving up
0x18	6	CMD_ARG	RW	The command's 32-bit argument
0x1C	7	CMD	RW	Index, response type, data flags. Write starts the operation
0x20	8	RESP0	RO	R1, or the low word of a longer response
0x24	9	RESP1	RO	R3 / R7 trailer
0x28	10	BLK_SIZE	RW	Bytes per block, up to <code>MAX_BLOCK_BYTES</code>
0x2C	11	BLK_COUNT	RW	Blocks in this transfer
0x30	12	DMA_ADDR	RW	Byte address in system memory
0x34	13	DMA_CTRL	RW	Mode. Only contiguous (0) is defined
0x38	14	DATA	RW	PIO window into the FIFO
0x3C	15	ERR_INFO	RO	Last tokens, last R1, phase that failed
0x40	16	CORE_INFO	RO	Build-time configuration

5.1 CTRL (0x00)

Bit	Name	Meaning
0	ENABLE	Master enable. 0 idles the sequencer
1	CS_MANUAL	Software drives <code>CS_n</code> directly
2	CS_VALUE	The level driven when <code>CS_MANUAL</code> is set
3	CRC_EN	Generate and check CRC16 on data blocks
4	DMA_EN	Data phase uses <code>m0</code> rather than the <code>DATA</code> window
5	CLK_RUN	Free-run the SPI clock with <code>CS_n</code> high
8	SRST_CMD	Soft reset, command path only
9	SRST_DAT	Soft reset, data path and FIFO only
10	SRST_ALL	Soft reset, everything except configuration

`CS_MANUAL` and `CLK_RUN` exist for initialisation: the specification requires at least 74 clocks with `CS` high before the first command, and software drives that directly rather than the sequencer inventing a pseudo-command for it.

5.2 STATUS (0x04)

Bit	Name	Meaning
0	CMD_BUSY	A command is in flight
1	DAT_BUSY	A data phase is in flight
2	DMA_BUSY	m0 has a transfer outstanding
3	CARD_BUSY	The card is holding MISO low
23:8	LEVEL	FIFO occupancy in bytes
24	CARD_PRES	From sd_cd_n , if USE_CARD_DETECT
25	CARD_WP	From sd_wp_n , if USE_CARD_DETECT
31	ERROR	Sticky OR of the error bits in IRQ_STATUS

5.3 IRQ_STATUS (0x0C) and IRQ_ENABLE (0x08)

Write 1 to clear. IRQ_ENABLE has the same layout.

Bit	Name	Meaning
0	CMD_DONE	Command and its response complete
1	DATA_DONE	The whole BLK_COUNT transfer is done
2	DMA_DONE	m0 has drained or filled the FIFO
8	ERR_CMD_TMO	No response within TIMEOUT
9	ERR_CMD_CRC	R1 reported a command CRC error
10	ERR_CMD_ILL	R1 reported an illegal command
11	ERR_DAT_TMO	No data token, or busy outlasted TIMEOUT
12	ERR_DAT_CRC	CRC16 mismatch on a read block
13	ERR_DAT_TOKEN	The card sent a data error token
14	ERR_WRITE	The data-response token rejected the block
15	ERR_DMA	m0 returned an error response

An interrupt handler must **read** IRQ_STATUS rather than infer the cause from the pin. During a data phase DMA_DONE raises the line repeatedly as bursts retire, so the pin being high says only that something happened.

5.4 CMD (0x1C)

Bit	Name	Meaning
5:0	INDEX	Command index
7:6	RESP	0 = R1, 1 = R1b, 2 = R2, 3 = R3/R7
8	DATA_EN	This command has a data phase
9	DATA_DIR	0 = card → host, 1 = host → card
10	MULTI	Stream BLK_COUNT blocks in hardware
11	AUTO_STOP	Terminate the stream without software
31	START	Write 1 to launch. Reads back as busy

The write is ignored while the sequencer is busy. That is correct — a second command must not corrupt a transfer in flight — but it means a driver that writes without checking loses the command silently, and polling afterwards does not catch it: busy is already clear, so the poll returns immediately for a command that never happened. Check `STATUS.CMD_BUSY` before writing, and confirm it goes high afterwards.

5.5 ERR_INFO (0x3C)

Bits	Meaning
7:0	Last data-response token
15:8	Last R1 byte received
23:16	Last data error token
27:24	The phase that timed out

The data-response token's shape is `xxx0sss1` — five bits of meaning in eight. Mask with `0x1F` before comparing. `0x05` is accepted (`sss = 010`), `0x0B` is a CRC error (`101`), `0x0D` is a write error (`110`).

6. Signals

6.1 Clock and reset

Signal	Direction	Width	Notes
clk	in	1	System clock. One domain — there is no SPI clock domain
reset_n	in	1	Active low, synchronous

The SPI clock is generated by strobes inside the same domain rather than as a second clock, so there is no clock-domain crossing anywhere in this core.

6.2 `csr` — Avalon-MM slave

Signal	Direction	Width
csr_address	in	CSR_ADDR_WIDTH (words)
csr_read	in	1
csr_write	in	1
csr_writedata	in	32
csr_byteenable	in	4
csr_readdata	out	32

6.3 `m0` — Avalon-MM master (`USE_DMA = 1`)

Signal	Direction	Width
m0_address	out	ADDR_WIDTH (bytes)
m0_read, m0_write	out	1
m0_writedata	out	32
m0_byteenable	out	4
m0_burstcount	out	M0_BURST_WIDTH
m0_waitrequest	in	1
m0_readdata	in	32
m0_readdatavalid	in	1
m0_response	in	2

`m0_byteenable` is always `0xF`. That is not laziness: Avalon permits the interconnect to suppress a read with all byteenables clear, which would hang the transfer with no error reported anywhere. An assertion enforces it, and `check_assertions_fire.sh` injects the fault to prove the assertion is alive.

6.4 sd — conduit

Signal	Direction	SD pin
sd_clk	out	CLK
sd_mosi	out	CMD
sd_miso	in	DAT0
sd_cs_n	out	DAT3/CS
sd_cd_n	in	socket switch (USE_CARD_DETECT)
sd_wp_n	in	socket switch (USE_CARD_DETECT)

7. Software

The HAL driver is at `HAL/src/altera_avalon_mm_sdcard_controller.c`, with the API in `HAL/inc/altera_avalon_mm_sdcard_controller.h`.

Function	Purpose
<code>alt_sdcard_init</code>	Identification at 400 kHz, capacity class, CSD and CID, then raise the clock
<code>alt_sdcard_read</code>	Read blocks into memory
<code>alt_sdcard_write</code>	Write blocks from memory
<code>alt_sdcard_block_count</code>	Card capacity in 512-byte blocks, parsed from the CSD
<code>alt_sdcard_command</code>	Issue an arbitrary command, for anything the API does not cover

7.1 Capacity, and why the CSD parse is separately tested

`alt_sdcard_block_count()` parses capacity out of the CSD, and the arithmetic is **different for the two structure versions** — not a field that moved. Reading a v2 card with the v1 formula yields a plausible number that is wrong by orders of magnitude, which is the kind of bug that survives casual testing and then corrupts a filesystem.

So it is unit-tested directly, against the exact CSD bytes the card model returns for each class, in `verification/check_driver_builds.sh` — the one piece of the driver the RTL regression cannot reach.

7.2 PIO mode

With `USE_DMA = 0`, `alt_sdcard_command()` is **non-blocking**: it issues, and you call the completion half yourself, moving words through the `DATA` window in between. It has to be. A blocking command call cannot service the window while the transfer runs, and on a write that is fatal — the sequencer reaches the data phase about ten byte-times after the command goes out.

8. Performance

8.1 Measured

16,696 SPI clocks to move 2,048 bytes, against a theoretical floor of 16,384 — **98.1% of line rate**. At a 25 MHz SPI clock that is 3.06 MB/s of a possible 3.125.

The regression **asserts** this cycle count rather than reporting it. A performance number nobody checks is a number that silently regresses.

8.2 Where it comes from

The shifter runs continuously with a one-deep prefetch, so the SPI clock does not pause between bytes. The remaining 1.9% is the protocol's own overhead — command frames, response polling, tokens, CRC bytes — not idle clocks.

8.3 What the DMA is and is not for

Not throughput. At SPI rates the memory side is never the bottleneck, and `M0_BURST_WIDTH = 1` measures the same as 8. What the DMA buys is **CPU time** and **immunity to interrupt latency**: without it the CPU must service the `DATA` window on a deadline for the whole of every block.

9. Verification

Everything that can be checked in software alone:

```
./verification/run_all.sh
```

Suite	What it does
Lint	Verilator <code>-Wall</code> across 10 parameter configurations
Simulation	3 testbenches, the full-core one in 5 configurations
Platform Designer	<code>hw.tcl</code> executed against stubbed Qsys commands — 22 checks
HAL driver	Compiled against stubbed Nios II headers, plus the CSD parse unit-tested
Assertions	3 faults injected, each required to be caught by the assertion meant to catch it
Facts	Every number in the documentation re-derived from source
CRC vectors	The polynomials checked against an independent Python model

9.1 The configuration sweep

The five configurations are not cosmetic variations:

Configuration	What only it reaches
<code>dma</code>	The reference case
<code>pio</code>	No master; software moves every word. The only one that reaches the data-phase stall timeouts
<code>sdsc</code>	Byte addressing. On an SDHC card the block-to-address conversion is the identity, so this is the only place it executes
<code>tight</code>	One block of buffer, so the data path refills mid-transfer
<code>noburst</code>	Single-beat Avalon transactions throughout

Adding this sweep immediately found a real defect: neither `S_RD_DATA` nor `S_WR_DATA` checked the timeout, so a data phase starved of data hung the core until a soft reset. With a master attached that cannot happen, which is exactly why it survived until the PIO configuration was run.

9.2 Assertions, and proving they are alive

24 bound SVA assertions and 5 cover points, in `tb/avalon_mm_sdcard_controller_sva.sv`.

They check invariants rather than results. That suits this core: most of its bugs were invisible on the wire — a byte queued twice, a CRC fed seven of eight bits, a transfer declared complete with a byte still in the shifter — and each one breaks an invariant you can state in a line.

A passing assertion proves nothing by itself, so `verification/check_assertions_fire.sh` injects three faults into scratch copies of the RTL and requires each to be caught by the assertion meant to catch it.

The first fault it tried turned out to be **unreachable**. `S_PRE_BUSY` clocks `0xFF` before every command, so the shifter is always already running when a sending state begins, and the `!hold_v` term in its idle output can never fire. The script records that rather than swapping in a fault that worked.

9.3 Questa

`simulation/questa/run_sim.tcl` runs the same sweep with coverage and non-vacuity reporting.

It has not been executed. Coverage and non-vacuity are the two things no other flow here provides, and they are worth having — but treat the first run as part of the work, not a formality. The SDRAM controller's equivalent file records two faults its own first run found, one of them RTL that Verilator linted clean and `vopt` rejected outright.

10. Limitations and known gaps

- **Never run on a board.** No physical card, no timing closure demonstrated, no board example. This is the largest gap and no amount of simulation closes it.
- **The card model is derived from the specification**, so it cannot reproduce the ways real cards deviate from it.
- **SPI mode only.** No 1-bit or 4-bit native SD, and no path to it in this design — SDHCI, the obvious standard to adopt for native mode, contains zero occurrences of "SPI" and cannot be retrofitted here.
- **No CMD12 for multi-block writes** beyond the `0xFD` stop token, which is what the specification requires; there is no `AUTO_STOP` equivalent issuing a separate command on the write side.
- `DMA_CTRL` **defines only contiguous mode.** The field exists so a descriptor mode could be added as a reserved encoding rather than an ABI break.
- **No erase, no lock/unlock, no SDIO.**
- **Questa flow untested** — see 9.3.

11. Revision history

Version	Change
v1.0	First issue.